Fuzzy Reward-based Reinforcement Learning for Clifford Circuit Synthesis

Giovanni Acampora[1], Allegra Cuzzocrea[1], Marco Lapegna[2], Roberto Schiattarella[1], Autilia Vitiello[1]
[1] *Department of Physics "Ettore Pancini"*, University of Naples Federico II, Naples, Italy
[2] *Department of Mathematics and Applications "Renato Caccioppoli"*, University of Naples Federico II, Naples, Italy
{giovanni.acampora, allegra.cuzzocrea, marco.lapegna, roberto.schiattarella, autilia.vitiello}@unina.it

***Abstract*—Quantum operator synthesis is a key task in quantum transpilers, enabling the translation of high-level algorithms into hardware-executable gate sequences by decomposing arbitrary operators into native gates while optimizing for efficiency and constraints. Recently, reinforcement learning (RL) has emerged as a promising approach for quantum circuit synthesis, particularly for Clifford operators. However, the crisp-reward functions used in current RL frameworks may fail to capture the subtleties of complex quantum circuit synthesis tasks, leading to suboptimal designs. In this paper, this gap is addressed by integrating a fuzzy reward into an RL algorithm to synthesize Clifford operators. The fuzzy reward is computed by a fuzzy rule-based system, designed from the analysis of the error rate in current IBM quantum devices. The experiments show that our fuzzy reward accelerates convergence, achieving the same performance in terms of depth and controlled-X (CX) depth as the crisp-reward algorithm, but with fewer training steps. This highlights the efficiency of fuzzy rewards and their potential to improve the synthesis capabilities of RL systems.**

***Index Terms*—Quantum Circuit Synthesis, Fuzzy Reward, Reinforcement Learning, Quantum Computing**

I. Introduction

Quantum computing has emerged as a transformative paradigm, providing potential solutions to problems that are intractable for classical computers. [1], [2]. Using the principles of quantum mechanics, quantum computers offer the potential to revolutionize fields such as cryptography, biology, database search, and so on. [3]. However, current quantum hardware is still within the Noisy Intermediate-Scale Quantum (NISQ) era, characterized by a limited number of qubits, high error rates, and imperfect gate operations. These limitations hinder the performance of large-scale quantum algorithms and highlight the need for effective strategies to optimize the execution of quantum algorithms on unreliable hardware. In this scenario, quantum transpilers can play a key role in converting complex quantum algorithms into an optimized set of low-level quantum gates tailored to specific hardware constraints and minimizing errors. One of the core activities in the transpilation process is quantum circuit synthesis, which aims to construct efficient gate-level implementations of unitary transformations. Recently, various machine learning (ML) techniques have been explored in this field [4] [5], but the challenge of obtaining sufficiently large labeled datasets limits their scalability for real-world applications. For this reason, IBM introduced a RL-based approach to train models for Clifford, Permutation, and Linear function synthesis, and to compute quantum circuits compatible with native device instruction sets and connectivity constraints [6]. This approach trains agents to interact with the quantum environment, exploring different gate sequences to incrementally build a quantum circuit that replicates the computational behavior of the input operator while considering performance metrics, such as circuit depth, and ensuring compatibility with a "specific" quantum hardware. This research led to the development of a series of activities based on the use of RL in circuit synthesis. For example, the approach presented [7], attempts to improve the performance of the IBM framework by allowing the RL synthesis to be used on different processor topologies and act as a flexible tool for adapting quantum operators to various hardware architectures, enhancing both efficiency and scalability across diverse quantum devices. However, both approaches suffer from a critical bottleneck: the RL reward policy relies on crisp values, making them unable to capture uncertainty and imprecision in synthesis steps, which can lead to suboptimal circuit synthesis and slower convergence. This paper bridges this significant gap in quantum transpilation by extending the RL synthesis approach presented in [7] with a reward function implemented by a Mamdani rule-based fuzzy system. These rules have been identified by carrying out a deep analysis of error rate in current generation of IBM quantum computers. The experiments show that the proposed fuzzy reward enables the circuit synthesis agent to achieve the same synthesis performance as the original crisp-reward-based algorithm but with significantly fewer training steps, making the proposed framework a good candidate to be embodied in the future generation of quantum transpilers. This result strongly highlights the effectiveness of integrating fuzzy logic into RL-based quantum circuit synthesis, facilitating faster convergence and more efficient learning.

This work was supported by the National Center for HPC, Big Data, and Quantum Computing (Spoke 1: Future HPC) in the framework of Next Generation EU program. Giovanni Acampora acknowledges financial support from the PNRR MUR project PE0000013-FAIR.

II. Related Work

Quantum circuit synthesis plays a fundamental role in quantum transpiling, particularly in the design and optimization of quantum circuits, which serve as the building blocks of quantum algorithms. Consequently, in recent years, various synthesis techniques have been proposed to accelerate the

transition toward the practical implementation of quantum computing. The first research effort in AI-driven quantum circuit synthesis is presented in [8], where A*, a heuristic search algorithm, is employed for topology-aware synthesis. The primary objective is to minimize the number of CNOT gates, while considering the connectivity constraints of the quantum processor. Later, the method was improved in [9], incorporating circuit partitioning prior to synthesis. Building on these early efforts, machine learning-based approaches for quantum circuit synthesis began to emerge, with a particular focus on RL. In [10], the authors combine Markov Decision Processes (MDP) and RL to address the problem of synthesis of quantum logic gates. In [11], deep RL is used to synthesize quantum circuits by addressing the high-dimensional and complex nature of quantum hardware, where the vast search space of possible gate sequences requires efficient exploration and optimization strategies to improve performance and resource utilization. In [12], the authors apply RL to optimize quantum transpiling for specific quantum devices, known as *topological quantum computing*, enhancing the efficiency of the gate sequence in fault-tolerant systems. Finally, the authors in [13] merge deep Q learning with a variant of A* to try to further improve the synthesis capabilities of RL. Recently, the focus of research on RL for circuit synthesis has changed to Clifford circuits. Clifford circuits are a subset of quantum circuits that are particularly important in quantum error correction and other quantum computing tasks. These circuits are often easier to handle analytically and play a crucial role in improving the reliability of quantum systems. In this context, IBM introduced a RL-based algorithm for Clifford circuit synthesis tailored to a specific processor topology [6]. While highly performative and innovative, this approach is limited by the need for a priori knowledge of the quantum processor topology, which restricts its applicability to scenarios where such information is readily available. For this reason, in [7] the authors present a more general RL-based approach for circuit synthesis where a technique known as invalid action masking allows training of a unique model capable of handling multiple topologies simultaneously. Although this approach achieves significant performance in synthesizing Clifford circuits for different quantum processors, the crisp nature of the reward function used in its reinforcement policy does not allow for properly managing the vagueness and imprecision that characterize the steps of the synthesis. This paper bridges this gap by proposing the design of a reinforcement policy based on a fuzzy reward to improve the performance of the framework described above. As shown in experiments, the use of fuzzy reward accelerates training of topology-aware RL models for circuit synthesis, while maintaining similar synthesis performance.

III. BACKGROUND

In this section, the fundamental concepts of quantum computing, with particular emphasis on Clifford circuits and quantum circuit synthesis, are introduced. In addition, the key principles of RL are explored.

A. Quantum Computing

Quantum computing uses the principles of quantum mechanics to process information in new ways. Unlike classical computers, quantum computers are based on qubits. A qubit can exist in a state of 0, 1, or in any quantum superposition of these two states. Mathematically, the state of a single qubit is expressed as:

$$|\phi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{1}$$

where α and β are complex numbers satisfying the condition $|\alpha|^2 + |\beta|^2 = 1$ and the square of their magnitudes represents the probability of measuring the qubit in 0 and 1 states. In a similar manner, the quantum state of a system consisting of N qubits can be expressed as follows:

$$|\psi\rangle = \sum_{i=0}^{2^N-1} \alpha_i\,|i\rangle \tag{2}$$

where $\alpha_i \in \mathcal{C}$ and its squared magnitude represents the probability of measuring the system in the classical state i. In quantum computing, quantum circuits are sequences of quantum gates, i.e., the atomic operations that manipulate quantum states, and they are used to implement quantum algorithms. A fundamental class of quantum circuits is represented by Clifford circuits. These circuits are defined by the Clifford group, which includes the H, S, and CX gates [14]. Additionally, a N-qubit quantum state corresponding to a Clifford circuit can be represented using a tableau, a $2N \times 2N$ boolean matrix paired with a $2N$-dimensional Boolean phase vector. Due to their structure, they can be efficiently simulated on classical probabilistic computers in polynomial time, as established by the Gottesman-Knill theorem [15]. Clifford circuits are foundational to numerous applications in quantum computing, such as the design of stabilizer codes for quantum error correction [1].

B. Quantum Transpiling and Circuit Synthesis

Quantum computers are made up of a limited set of gates and a specific topology. For this reason, quantum compiling, also known as transpiling, is essential for the execution of quantum algorithms in real devices. Quantum transpiling is the process of transforming a high-level quantum algorithm into a low-level circuit tailored for execution on a specific quantum hardware platform. Furthermore, NISQ devices are characterized by quantum noise. For this reason, a good transpiler is able to find efficient and noise-tolerant representations of quantum algorithms. A crucial step of quantum transpiling is quantum circuit synthesis, which refers to the process of constructing quantum circuits that implement a desired quantum operator. This involves translating a mathematical description of the desired unitary operation into a sequence of quantum gates that can be executed on a quantum computer.

C. Reinforcement Learning

Reinforcement Learning (RL) is a paradigm of machine learning where an agent learns to make decisions by interacting with an environment. In particular, the agent explores

the environment by taking actions and receiving feedback, in the form of rewards, to improve its future actions. In fact, its goal is to maximize cumulative rewards. A fundamental concept in RL training is the *episode*. An episode begins when a new input is provided to the agent and ends when a success condition is met or a maximum number of steps is reached. A popular RL algorithm is Proximal Policy Optimization (PPO). It uses a surrogate objective function to improve the policy while preventing large and destabilizing updates. PPO training is composed of multiple rollouts, that is, collections of episodes where experiences are gathered, in the form of state-action-reward tuples. After each rollout, the agent uses the collected data to update its policy.

IV. The Fuzzy-based Synthesis Approach

In this section, the Clifford synthesis process used in previous works [6], [7] is described with a particular focus on the crisp reward proposed within that framework. Following this, a comprehensive description of our proposed fuzzy reward system is introduced which has been incorporated into the original framework to analyze and evaluate its performance. Finally, the training procedure is thoroughly explored.

A. RL-based Synthesis Process

In [6] the Clifford synthesis problem has been modeled as a sequential decision process. In particular, the initial state of this process is represented by the tableau $\mathcal{T}$, which provides a matrix-based description of the target operator U associated with a N-qubit circuit $\mathcal{QC}$. The action space of the RL environment is composed of all Clifford gates that can be applied to the N qubits. In particular, $\mathcal{AS} = \{H_i, S_i, CX_{ij}\}_{i,j\in[0,N-1]}$. In this set, H_i and S_i represent the Hadamard and Phase gates applied to the i-th qubit, respectively, while CX_{ij} denotes a CX gate, with the i-th qubit as the control and the j-th qubit as the target. Since the resulting synthesized circuit must respect the connectivity constraints of a given topology $\mathcal{P}$, in [7] the invalid action masking technique was introduced providing to the agent $\mathcal{A}$ a subset of $\mathcal{AS}$, $\mathcal{AS}_\mathcal{P}$, in which only the CX gates CX_{ij} compatible with $\mathcal{P}$ are available. Therefore, at each step, $\mathcal{A}$ chooses an action from $\mathcal{AS}_\mathcal{P}$. This action is applied to $\mathcal{QC}$, obtaining the new tableau, which represents the new state. The process continues until $\mathcal{T}$ becomes the identity matrix, which means that $\mathcal{A}$ has correctly implemented $\mathcal{U}$, except for a phase factor. A key element that plays a critical role in guiding the learning process is the reward $\mathcal{R}$. Specifically, it serves as the feedback signal that informs the agent about the quality of its actions in a given state. For this reason, the design of an efficient reward function is fundamental in RL-based algorithms. The crisp original reward used in [7] is reported below:

$$\mathcal{R} = \begin{cases} 1000 - \\ D_i(D_i - D_{i-1}) - c_0 Tq_i & \text{if Success} \\ \\ \mathcal{ID} - c_1(D_i - D_{i-1}) - \\ c_2(Tq_i - Tq_{i-1}) - & \text{if not Success} \\ c_3(Sq_i - Sq_{i-1}) \end{cases} \tag{3}$$

In particular, D_i is the depth of the synthesized circuit after the i-th action has been applied; Tq_i represents the number of two-qubit gates and Sq_i the number of single-qubit gates.The inverse distance $\mathcal{ID}$ is in range $[0, 1]$ and represents the similarity between the $2N \times 2N$ identity matrix I and the $2N \times 2N$ matrix M describing the tableau at the i-th step except for the phase factor. In particular, it is computed as follows:

$$\mathcal{ID} = 1 - \frac{\sum_{lm} I_{lm} - M_{lm}}{2N \times 2N} \tag{4}$$

where the subscripts l and m indicate the l-th row and the m-th column of the respective matrices. This reward is designed to assume an increasingly higher value as the current operator approaches I and assumes a very high positive value when $\mathcal{A}$ achieves success. Furthermore, the increase in depth, especially the depth of the CX, is penalized according to the selected coefficients c_0, c_1, c_2, and c_3.

B. Fuzzy reward design

To overcome the limitation of crisp reward in RL-based synthesis, a Mamdani fuzzy logic controller (FLC) is designed and integrated into the reward function of the RL algorithm for Clifford synthesis presented in Section IV-A. The FLC is composed of two input variables and one output variable representing the reward related to the action of the agent. In detail, the two input variables are the Jaccard similarity between the identity matrix and the matrix representing the Clifford Tableau after the action of the agent, and the CX count of the synthesized quantum circuit. The Jaccard similarity is a commonly used metric to assess the similarity between two sets of matrices and is particularly useful when dealing with binary matrices. The formal description of the Jaccard similarity for two matrices A and B is as follows:

$$S_J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{5}$$

Therefore, this similarity ranges from 0 to 1, reaching a value of 1 when the two matrices are identical. Moreover, using just the CX count as the second input variable for computing the reward is justified by the fact that two-qubit gates exhibit significantly higher noise levels compared to single-qubit gates on current quantum devices, making them the primary source of errors in current quantum hardware. In particular, it has been observed that IBM devices exhibit an average error rate of 10^{-2} for two-qubit gates. This means that on average one error occurs for every 100 two-qubit gates. Hence, the objective is to design a fuzzy system that penalizes CX counts that approach or exceed the 100 threshold. In particular, for the similarity input variable, five linguistic terms are defined in the range $[0, 1]$: Very Low, Low, Medium, High, and Very High. Differently, the CX count input is represented using four linguistic terms in the range $[0, 100]$: Very Low, Low, Medium, and High. On the other hand, the output variable of the FLC is the reward, which is modeled in the range $[0, 1]$ and represented through five linguistic variables: Very Low, Low, Medium, High, and Very High. The two input variables

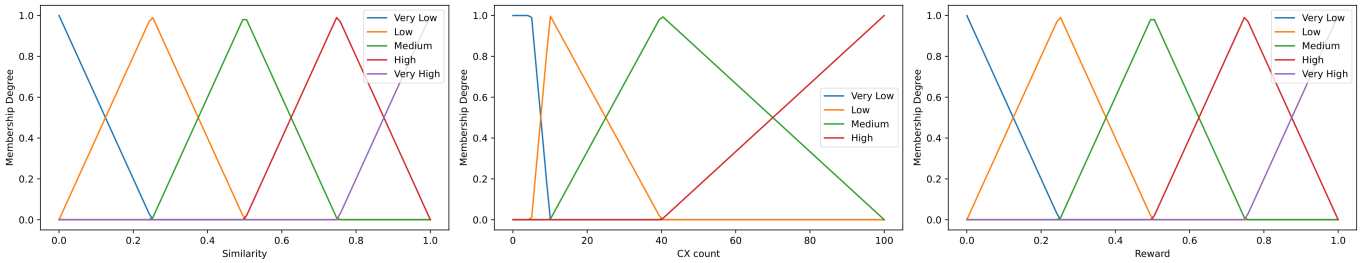


Fig. 1. Fuzzy set partitions for similarity, CX count, and reward.

TABLE I
RULE FORMULATION OF THE FUZZY REWARD FOR QUANTUM CIRCUIT SYNTHESIS.

S \ CX	**Very Low**	**Low**	**Medium**	**High**
Very Low	Very Low	Very Low	Very Low	Very Low
Low	Low	Low	Low	Very Low
Medium	Medium	Medium	Medium	Medium
High	High	High	High	High
Very High	Very High	Very High	Very High	High

and the output variable have been partitioned as shown in Fig. 1. The rules defined for the Clifford synthesis algorithm, are summarized in Table I, where the similarity is indicated as S and the CX count as CX. In detail, the rules generally prioritize the similarity input, as it is the key factor that drives the agent toward success. However, in specific cases, the reward is adjusted based on the CX count input. For instance, a high similarity combined with a high CX count is not rewarded as much as when the CX count is very low, low, or medium. Furthermore, a low similarity coupled with a high CX count indicates a malfunction in the synthesis process, resulting in a very low reward. The goal is to guide the agent in learning to synthesize circuits correctly, while simultaneously penalizing a high CX count. In addition, upon success, the reward does not depend on the FLC anymore, instead it takes a fixed value of 1000. This approach provides a strong incentive for the agent to prioritize reaching the desired outcome, reinforcing successful behaviors, and speeding up the training. In all the experiments performed, the FLC was equipped with min t-norm and a centroid-based defuzzification approach.

C. Training process

The training process is characterized by the use of the PPO algorithm, along with the parallel execution of multiple vectorized environments, which helps accelerate convergence. Furthermore, a curriculum learning approach with two distinct training phases was adopted, as in [6], [7]. In the first phase, a difficulty parameter d is introduced and dynamically increases throughout the process. Clifford operators are generated from N-qubit random quantum circuits, each consisting of d Clifford gates. For each operator, P is randomly selected from a predefined set of topologies. The value of d starts at 1 and increases whenever the ratio of successful episodes to the total

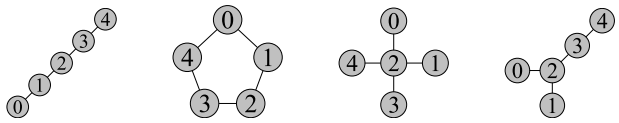


Fig. 2. 5-qubit topologies used during the training.

number of episodes exceeds a predefined threshold. Due to the use of invalid action masking, A is restricted to choosing actions only from $\mathcal{AS}^{\mathcal{P}}$, which is constructed based on P. This approach allows the model to generalize across multiple topologies, requiring training only once. Each episode ends either when success is achieved or a number of steps equal to d are reached. This ensures that the number of gates and consequently the depth of the resulting circuit are minimized. In the second phase, target operators are generated directly from the N-qubit dimensional Clifford space to make the agent capable of synthesizing any Clifford operator. The fuzzy reward system, as described in Section IV-B, was integrated into the entire training process to guide the agent toward more optimal synthesis of quantum circuits.

V. EXPERIMENTS AND RESULTS

This section presents the experiments conducted to train two 5-qubit models: one called CR, using the original crisp-reward-based approach, as reported in [7], and the other called FR, adopting the fuzzy-reward-based approach proposed in this paper. A comparative analysis follows, evaluating the two models trained for the same number of steps in terms of training speed. A focus on the first part of the training is reported as it is considered meaningful for the comparison. Finally, the performance in terms of circuit depth and CX depth of the two models is discussed.

A. Training setup

To ensure a fair comparison between the original and fuzzy reward approaches, both models were trained using the same setup as described in [7]. Specifically, training was conducted using the MaskablePPO class from Stable Baselines3 [16], with the default multilayer perceptron (MLP) policy. The Gymnasium interface [17] was employed to implement the environment and create 40 parallel vectorized environments to speed up the training. Fig. 2 reports the 5-qubit topologies

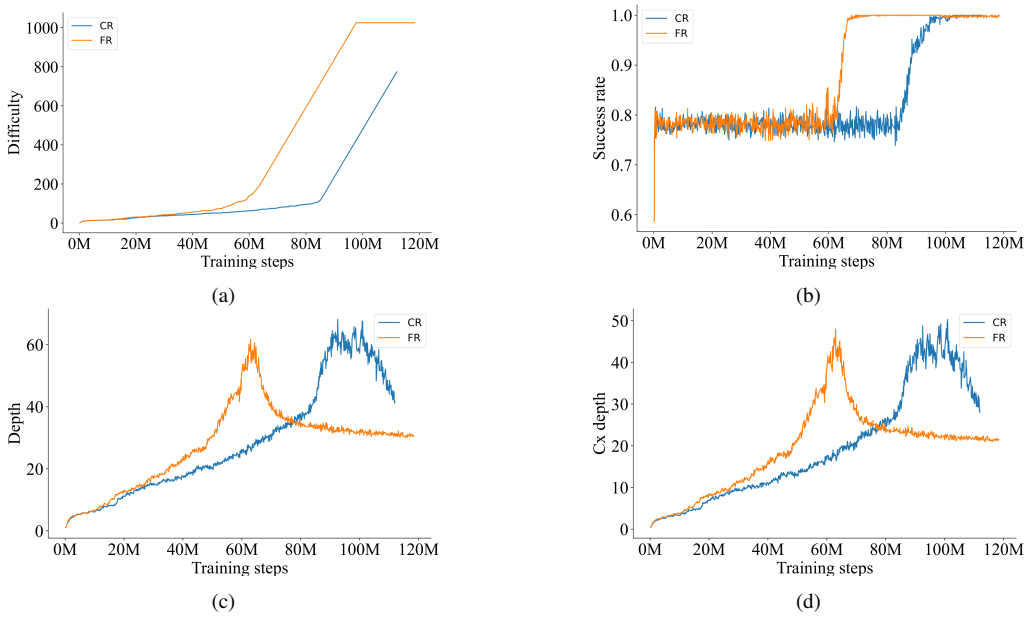


Fig. 3. Training progress of CR and FR, showing the evolution of key quantities over training steps. In particular, Fig. (a) reports the difficulty, Fig. (b) the success rate, Fig. (c) the depth, and Fig. (d) the CX depth.

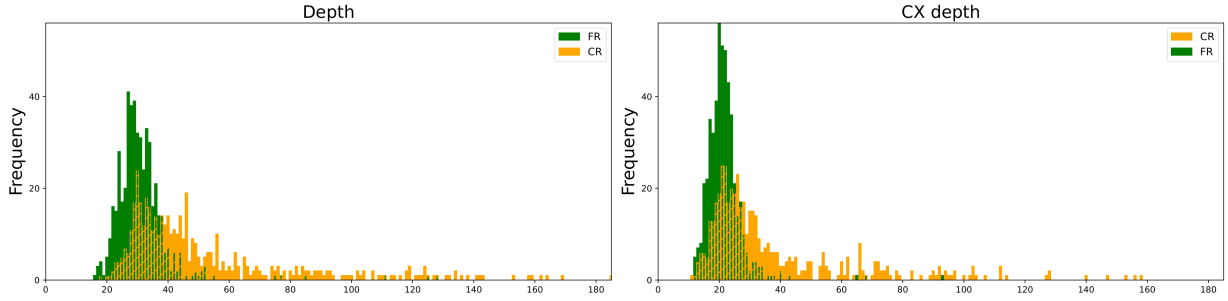


Fig. 4. Performance comparison between CR and FR in terms of depth and CX depth.

considered during the training. The parameter d increases from 1 to 1024 each time the threshold, set to 0.8, is reached. Once d assumes 1024, the target operators are sampled directly from the Clifford space through the method reported in [18]. Finally, the reward coefficients for CR were set as follows: $c_0 = 10$, $c_1 = 0.5$, $c_2 = 0.15$, and $c_3 = 0.1$.

B. Analysis of the training speed

This section analyzes the comparison of the training progress between the two models in the range of timesteps reported, with a focus on evaluating the differences in their training speeds. Fig. 3 reports the training progress of the two models in terms of difficulty, success rate, depth, and CX depth. As can be observed, the two models exhibit similar trends. In particular, both curves related to the success rate show an initial increase up to the threshold of 0.8, indicating that the agent is learning to synthesize operators generated with a difficulty level of 1. Then, the curves stabilize at 0.8 before rising to 1. Stabilizing at 1 means that the models are capable of synthesizing any operator from the Clifford space. Moreover, after a slower initial phase, the difficulty increases linearly up to 1024. From this point onward, this parameter remains constant as the operators are generated directly from the Clifford space. Both the depth and the CX depth show an initial increase and then, once the success rate reaches 1, they begin to decrease, leading to improved performance. However, an important consideration is that FR reaches a success rate of 1 significantly earlier than CR. In fact, FR is able to synthesize any Clifford operator after approximately 70 million training steps, while CR achieves this capability after around 100 million steps. Moreover, during the training progress reported, CR has not yet reached a difficulty level of 1024, while FR has. The difference of about 30 million steps is significant as it substantially reduces the computational effort required. An analysis of the depth and CX depth curves reveals that, after 120 million training steps, FR produces synthesized circuits with significantly lower depth and CX depth compared to CR, indicating that it exhibits more optimized performance.

C. Testing

This section discusses the performance evaluation of FR and CR, trained for the same number of timesteps, in terms of depth and CX depth. In particular, a test dataset composed of 500 random operators from the Clifford space was created. Furthermore, a topology randomly sampled from the available set was chosen and assigned to each operator. The topologies were evenly distributed within the dataset to ensure that the models were fairly tested in synthesizing operators across all available topologies. The comparison of the performance of the two models is reported in Fig. 4. First, FR was able to synthesize 100% of the dataset, while CR the 96%. It can be seen that FR provides synthesized circuits with a notably lower depth and CX depth with respect to CR. In fact, the values for CR extend for a range much larger than FR. The difference in performance is also obvious by computing the depth average and the CX depth averages. In fact, FR reports a depth average and a CX depth average of 31 and 21, respectively, which are significantly lower than the 50 and 35 reported for CR. In particular, FR reduces the depth average of 32% and the CX depth average of 30% compared to CR. Furthermore, the training of FR was extended to the number of timesteps of the crisp-reward-based model presented in [7], and it stabilizes at the same performance level as the crisp counterpart while maintaining a significant gap in terms of steps. In fact, FR reaches the same optimal performance as shown in [7], overcoming the Qiskit's SDK synthesizers, in a significantly smaller number of training steps than CR. These results demonstrate the efficacy of the fuzzy reward proposed, highlighting the fact that the reward, despite considering only the number of CX gates and not the number of single-qubit gates or the depth, is still able to guide the agent toward generating circuits with optimal depth and CX depth. Finally, the test times for FR were found to be of the same order of magnitude as those of CR. This indicates that FR offers considerable efficiency, enabling the same performance to be achieved with less effort.

VI. Conclusion

This paper improves the Clifford circuit synthesis mechanism based on RL [7] by introducing an innovative fuzzy reward to train synthesis agents. Specifically, integrating fuzzy logic enhanced the effectiveness of a key RL element, the reward, as it directly impacts the agent's learning and decision-making in quantum operator synthesis. A PPO model, using this new methodology, was successfully trained to synthesize 5-qubit Clifford operators on particular processor topologies. The experiments demonstrate that the fuzzy reward mechanism allows the agent to perform at the same level as the traditional crisp-reward-based algorithm but requires considerably fewer training steps. This reduction in training steps considerably reduces computational costs, demonstrating the effectiveness of fuzzy rewards in improving learning efficiency. A future step will involve the automatic learning of fuzzy rules from the data collected during the training of the version without fuzzy reward, along with an analysis of the interpretability of the derived rules, ultimately aiming to build an interpretable RL model. Integrating fuzzy logic with RL addresses key challenges in quantum circuit synthesis, offering a more adaptable and efficient framework. This advancement paves the way for research into combining fuzzy logic with RL to develop efficient quantum transpilers and accelerate progress toward fault-tolerant quantum computing.

References

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2011.

[2] F. Arute et al., "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, no. 7779, pp. 505–510, Oct. 2019.

[3] A. M. Dalzell, S. McArdle, M. Berta, P. Bienias, C.-F. Chen, A. Gilyén, C. T. Hann, M. J. Kastoryano, E. T. Khabiboulline, A. Kubica, G. Salton, S. Wang, and F. G. S. L. Brandão, "Quantum algorithms: A survey of applications and end-to-end complexities," 2023. [Online]. Available: https://arxiv.org/abs/2310.03011

[4] M. Weiden et al., "Improving quantum circuit synthesis with machine learning," in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2023, pp. 1–11.

[5] K. Murakami and J. Zhao, "Automated synthesis of quantum circuits using neural network," in *Proceedings - 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security, QRS 2022*, ser. IEEE International Conference on Software Quality, Reliability and Security, QRS. United States: Institute of Electrical and Electronics Engineers Inc., 2022, pp. 694–702.

[6] D. Kremer, V. Villar, H. Paik, I. Duran, I. Faro, and J. Cruz-Benito, "Practical and efficient quantum circuit synthesis and transpiling with reinforcement learning," 2024. [Online]. Available: https://arxiv.org/abs/2405.13196

[7] G. Acampora, A. Cuzzocrea, M. Lapegna, R. Schiattarella, and A. Vitiello, "Generalizing reinforcement learning-based quantum circuit synthesis across multiple topologies," *Under publication in proceedings of IEEE SSCI 2025*, 2025.

[8] M. G. Davis, E. Smith, A. Tudor, K. Sen, I. Siddiqi, and C. Iancu, "Towards optimal topology aware quantum circuit synthesis," in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2020, pp. 223–234.

[9] M. Weiden, J. Kalloor, J. Kubiatowicz, E. Younis, and C. Iancu, "Wide quantum circuit optimization with topology aware synthesis," in *2022 IEEE/ACM Third International Workshop on Quantum Computing Software (QCS)*, 2022, pp. 1–11.

[10] M. S. Alam, N. F. Berthusen, and P. P. Orth, "Quantum logic gate synthesis as a Markov decision process," *npj Quantum Information*, vol. 9, no. 1, p. 108, Oct. 2023.

[11] L. Moro, M. G. A. Paris, M. Restelli, and E. Prati, "Quantum compiling by deep reinforcement learning," *Communications Physics*, vol. 4, no. 1, p. 178, Aug. 2021.

[12] Y.-H. Zhang, P.-L. Zheng, Y. Zhang, and D.-L. Deng, "Topological quantum compiling with reinforcement learning," *Phys. Rev. Lett.*, vol. 125, p. 170501, Oct 2020.

[13] Q. Chen, Y. Du, Y. Jiao, X. Lu, X. Wu, and Q. Zhao, "Efficient and practical quantum compiler towards multi-qubit systems with deep reinforcement learning*," *Quantum Science and Technology*, vol. 9, no. 4, p. 045002, jul 2024.

[14] S. Aaronson and D. Gottesman, "Improved simulation of stabilizer circuits," *Phys. Rev. A*, vol. 70, p. 052328, Nov 2004.

[15] D. Gottesman, "The heisenberg representation of quantum computers," 1998. [Online]. Available: https://arxiv.org/abs/quant-ph/9807006

[16] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[17] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," 2016. [Online]. Available: https://arxiv.org/abs/1606.01540

[18] S. Bravyi and D. Maslov, "Hadamard-free circuits expose the structure of the clifford group," *IEEE Transactions on Information Theory*, vol. 67, no. 7, p. 4546–4563, Jul. 2021.